

TAREA 01 — BASE DE DATOS Y SUPABASE

Fase 2 · Etapa 0 · 27 de agosto de 2026 **Lee primero:** 00-LEEME-PRIMERO.md de esta misma carpeta.

□ POR QUÉ ESTA TAREA VA PRIMERO

Decisión del 27/08. El plan original era construir las pantallas con datos falsos y conectar la base después. **Se cambió, y por tres razones:**

1 · El esquema ya existe. Las 22 tablas están definidas en 03-SISTEMA/schema.sql. Inventar formas de datos falsos cuando la forma real ya está decidida es trabajo doble: primero se escribe el maqueteo, después se descubre que no coincide y se reescribe la pantalla.

2 · Los tipos se generan solos. Supabase produce los tipos de TypeScript desde el esquema. El frontend recibe tipos reales, no objetos escritos a mano.

3 · El MCP permite verificar de verdad. Con Supabase conectado, se puede consultar la base directamente y comprobar qué contiene el sistema — no lo que el código dice que contiene.

Esa tercera razón es la más importante. En la Fase 1, Antigravity declaró "32 de 32 pruebas pasadas" con la aplicación rota. La única verificación posible era leer el código. **Con el MCP, se consulta la base y se ve la verdad.**

□ ANTES DE ESCRIBIR UNA LÍNEA

En la Fase 1 corrompiste archivos NUEVE veces

Archivos cortados a media palabra, otros rellenos de bytes nulos. **Todos tus commits estuvieron limpios — lo que se dañaba era el directorio de trabajo.**

```
bash git init git add -A git commit -m "inicio fase 2 - tarea 01"
```

Commit después de CADA bloque. Y antes de decir que terminaste:

```
bash grep -rLP '\x00' . --include="*.sql" --include="*.ts" && echo "☐ BYTES NULOS" git status --short
```

Si un archivo se corrompe: `git show HEAD:archivo > archivo` (*git restore falla en esta unidad con "Operation not permitted"*)

QUÉ SE CONSTRUYE ACÁ

- ☐ Proyecto Supabase creado y conectado
- ☐ Las 22 tablas de 03-SISTEMA/schema.sql
- ☐ Las filas de configuración del plan – ni un porcentaje en código
- ☐ Los productos y packs reales
- ☐ Las escalas de comisión, incluida la excepción del Kit
- ☐ Políticas de seguridad por fila (RLS)
- ☐ Tipos de TypeScript generados

☐ Lo que NO se construye acá

- ☐ Pantallas TAREA-02
- ☐ Red simulada de 500 socios TAREA-03
- ☐ Motor de comisiones TAREA-04
- ☐ Cualquier lógica de negocio en la base

El motor NO va en la base de datos. Nada de funciones almacenadas que calculen comisiones. El motor es TypeScript puro y se prueba sin base. *Artículo VII de la constitución: aislamiento del motor.*

BLOQUE A · PROYECTO SUPABASE

- ☐ Crear el proyecto – región más cercana a Perú
- ☐ Guardar las credenciales en `.env.local`
- ☐ `.env.local` en `.gitignore` ☐ nunca se sube al repositorio
- ☐ Instalar el CLI de Supabase
- ☐ `supabase init` → carpeta de migraciones

Las variables

```
VITE_SUPABASE_URL=  
VITE_SUPABASE_ANON_KEY=
```

SUPABASE_SERVICE_ROLE_KEY= ☐ SOLO en el servidor

La service_role nunca va al frontend. Salta todas las políticas de seguridad. Si aparece en el navegador, cualquiera puede leer y escribir toda la base.

Verificación: `grep -rn "service_role" src/` debe salir vacío.

Commit.

BLOQUE B · LAS 22 TABLAS

Fuente: 03-SISTEMA/schema.sql. **No inventes tablas ni cambies nombres.**

☐ LOS NOMBRES REALES — no los inventes

config la configuración del plan (57 filas) nivel_comision las escalas: tipo 'patrocinio' | 'residual' pack_comision_especial la excepción del Kit comision los movimientos pagados producto pack socio red_ancestro ciclo orden orden_detalle voucher movimiento_puntos punto_entrega envio activacion rango rango_ciclo wallet_movimiento solicitud_retiro periodo_global auditoria

Las columnas de dinero terminan en _cent, no en _centimos:

producto.precio_lista_cent pack.precio_cent comision.monto_cent wallet_movimiento.monto_cent

Se aplica como migración, no a mano en el panel:

```
supabase migration new esquema_inicial
# pegar schema.sql
supabase db push
```

Las reglas del esquema que no se negocian

1 · El dinero en enteros.

monto_cent	BIGINT NOT NULL	☐
monto	NUMERIC(10,2)	☐ nunca

S/. 7.20 se guarda como 720.

2 · Los libros son de solo-agregar.

Las tablas comision y wallet_movimiento **no** llevan UPDATE ni DELETE. Si hay un error, se agrega un movimiento que lo corrige.

```
-- prohibido modificar el histórico
REVOKE UPDATE, DELETE ON comision FROM authenticated;
REVOKE UPDATE, DELETE ON wallet_movimiento FROM authenticated;
REVOKE UPDATE, DELETE ON auditoria FROM authenticated;
```

3 • Toda tabla lleva creado_en y las que se puedan modificar, actualizado_en.

4 • Sin ciclos en el árbol. Una restricción que impida que un socio sea su propio ancestro.

Verificación

```
SELECT count(*) FROM information_schema.tables WHERE table_schema='public';
-- debe dar 22

-- ninguna columna de dinero puede ser decimal
SELECT table_name, column_name, data_type
FROM information_schema.columns
WHERE (column_name LIKE '%_cent' OR column_name LIKE '%monto%')
AND data_type NOT IN ('bigint', 'integer');
-- debe salir vacío
```

Commit.

BLOQUE C • LA CONFIGURACIÓN DEL PLAN

Este bloque es el corazón de la tarea.

Ni un solo porcentaje escrito en el código

Todos los valores del plan viven en tablas y se leen de ahí. **Si mañana Máximo cambia un número, se cambia una fila.**

Artículo IV de la constitución: configuración sobre código.

Escala de patrocinio — 7 niveles, suman 30.8%

Nivel 1	40.0 %	Nivel 5	3.0 %
Nivel 2	20.0 %	Nivel 6	2.0 %
Nivel 3	10.0 %	Nivel 7	1.0 %
Nivel 4	5.0 %		

□ La excepción del Kit Emprendedor

```
INSERT INTO pack_comision_especial VALUES ('EMPRENDEDOR', 1, 41.700);
```

El Kit NO usa la escala de 7 niveles. Paga 41.7% al nivel 1 y nada más. Para un Kit de S/. 120 son S/. 50 al patrocinador directo; los niveles 2 al 7 no reciben nada.

Esto parece un error y no lo es. Antes de "corregirlo", lee el Artículo XIV de la constitución.

Escala residual — 10 niveles, 97% de los puntos

1 punto = S/. 1.00. Fijo, no se discute.

Ojo con el 4.167 — está en config, pero NO es para comisiones

```
sql ('valor_punto_soles', '4.167', 'decimal', 'Razón precio/punto. Se usa para reportes de volumen, NO para comisiones')
```

Se carga porque el schema lo trae, con esa descripción. Sale de dividir el precio de socio entre los puntos ($S/.75 \div 18$). **El motor NO lo puede usar nunca.** Para comisiones, 1 punto = S/. 1.00.

Las 57 filas de configuración

Vienen todas en schema.sql. No las escribas a mano ni inventes claves. Las críticas son:

```
puntos_activacion_mensual ..... 70
rango_baja_no_cobra ..... true
puntos_grupales_incluyen_personales ..... false
pack_hereda_puntos_producto ..... false
linea_estirada_max_pct ..... 50
precio_guardado_es ..... publico
monto_minimo_retiro_cent ..... 10000
```

Fuente completa: 03-SISTEMA/schema.sql y 01-NEGOCIO/01-PLAN-DE-COMPENSACION.md

Verificación

```
-- la escala de patrocinio suma 30.8
SELECT sum(porcentaje) FROM nivel_comision WHERE tipo = 'patrocinio';
-- 30.800

-- la excepción del Kit
SELECT * FROM pack_comision_especial;
```

```
-- EMPRENDEDOR, 1, 41.700

-- la configuración cargada
SELECT count(*) FROM config;
-- 57

-- los precios son los PÚBLICOS, no los de socio
SELECT codigo, precio_lista_cent, puntos FROM producto ORDER BY orden;
-- CAFE debe dar 15000, NO 7500
```

Commit.

BLOQUE D · PRODUCTOS Y PACKS

Fuente: 01-NEGOCIO/02-CATALOGO-PRODUCTOS-Y-PACKS.md

□ Los precios se guardan al PÚBLICO

```
Café
producto.precio_lista_cent .... 15000    ← S/. 150
producto.puntos ..... 18
```

Ya vienen corregidos en el schema.sql del 28/08. Antes el INSERT traía los precios de socio (7500) y un UPDATE al final los duplicaba — pero ese UPDATE apuntaba a precio_cent, columna que es de pack, no de producto. **Fallaba y cortaba la migración.** Ya no existe.

El precio de socio NO se guarda. Se calcula con el descuento_recompra_pct del pack que tiene el socio.

Si guardas S/. 75 como precio, la empresa cobra la mitad en cada recompra. Ese error casi entra al sistema. *Artículo XVIII.*

Los puntos son fijos

El descuento nunca los prorratea. El café da 18 puntos, se pague S/. 150, S/. 90 o S/. 75.

Los packs

Cada pack lleva su descuento_recompra_pct y SUS puntos_rango.

```
Emprendedor ... 40 % de recompra
Ejecutivo ..... 50 % de recompra (dentro del pack fue 40 %)
```

Gold 50 %
Empresarial ... 50 %

El Pack VIP no existe. No lo cargues.

Commit.

BLOQUE E · SEGURIDAD POR FILA (RLS) □

Sin esto, cualquier socio puede leer los datos de todos.

- ❑ RLS activado en TODAS las tablas, sin excepción
- ❑ El socio solo ve SUS pedidos, SUS comisiones, SU perfil
- ❑ El socio ve su red HACIA ABAJO, nunca hacia arriba ni de otra rama
- ❑ El socio NUNCA ve teléfono ni correo de su red
- ❑ El socio NUNCA ve las comisiones de otros
- ❑ Solo el administrador escribe pedidos y confirma pagos
- ❑ Nadie puede modificar ni borrar movimientos de comisión

El filtro del árbol va en el servidor, no en el navegador. Si la consulta trae toda la red y React esconde lo demás, el dato ya viajó — cualquiera lo ve en las herramientas del navegador.

Es dato personal ajeno. Ley 29733.

Fuente: 03-SISTEMA/05-ROLES-Y-PERMISOS.md

Pruebas obligatorias de este bloque

- ❑ Socio A intenta leer los pedidos de Socio B → 0 filas
- ❑ Socio A intenta leer la red de Socio B → 0 filas
- ❑ Socio A intenta ver el correo de su propia red → columna vacía
- ❑ Un socio intenta insertar un pedido → denegado
- ❑ Alguien intenta borrar un movimiento → denegado

No basta con escribir las políticas. Hay que intentar violarlas y que falle.

Commit.

BLOQUE F · TIPOS Y CLIENTE

```
supabase gen types typescript --local > src/tipos/supabase.ts
```

- ❑ Los tipos generados, no escritos a mano
- ❑ Un cliente único de Supabase, no uno por archivo
- ❑ Un comando en package.json para regenerar los tipos

Regenerar los tipos cada vez que cambie el esquema. Si el esquema y los tipos se separan, el frontend miente y TypeScript no avisa.

Commit.

BLOQUE G · PRUEBAS

Vitest

- ❑ El cliente conecta
- ❑ Las 22 tablas existen
- ❑ Los porcentajes de patrocinio suman 30.8
- ❑ El Kit tiene su comisión especial cargada
- ❑ Los precios guardados son los públicos, no los de socio
- ❑ Ninguna columna de dinero es decimal

Pruebas de RLS — las más importantes

- ❑ Los 5 intentos de violación del Bloque E fallan

Integridad

```
grep -rn "service_role" src/          # VACÍO
git status --short                    # limpio
```

Commit.

ORDEN

``` 0º git init + commit 1º Bloque A · proyecto Supabase commit 2º Bloque B · las 22 tab